GitHub Org Security & Access-Hygiene Automation — submission summary

Makilesh · SDE/Automation intern task · github.com/Makilesh/Github_Org_Security

Scans every repository in a GitHub organization, reports its security advisories, measures how much each collaborator actually contributes to each repo, and **suggests** access removals. It never revokes anything.

Contribution scoring, and why

```
activity = 3.0·ln(1+commits) + 2.5·ln(1+reviews) + 2.0·ln(1+PRs merged) + 1.0·ln(1+PRs opened)
score    = activity × 0.5^(days since last activity / 180)
risk     = permission weight / (1 + score)        admin 3 · maintain 2 · write 1.5 · triage/read 0.5
```

Flagged below **score 5.0**; suggestions ranked by **risk**, highest first.

- **Logarithmic**, because the gap between 0 and 5 commits matters and the gap between 200 and 205 does not.
- **Reviews weighted 2.5, nearly as high as commits** — this is the most consequential number here. A naive commit count flags exactly the wrong person: the senior engineer who reviews constantly, writes little code, and holds the most load-bearing access on the team. A dedicated test asserts that person is never flagged.
- **Half-life decay, not a cutoff**, because a hard 180-day boundary makes someone at 179 days safe and at 181 days flagged — indefensible in the conversation that follows.
- **Risk divides power by usage**, because a reviewer triaging a list needs "inactive *and* dangerous", not just "inactive". A dormant admin outranks a dormant reader.

Assumptions and edge cases

Handled explicitly, each with a test: empty repos (409), archived repos (**scanned, not skipped** — that access is still live, and archiving only changes the remediation advice), forks (skipped, configurable), bots, org owners, brand-new repos, single-contributor repos, custom repository roles (weighted as *write*, never as harmless).

Verified on a live organization, not only on fixtures: 7 repositories, one org-wide call returning 32 Dependabot alerts (15 critical or high), and 2 access suggestions out of 21 grants assessed — while correctly leaving alone an active admin, two org owners, a member whose read access comes from an org-wide setting, and a contributor holding no access at all. Running against real data also exposed four bugs that fixtures could not — a refused access listing rendering as "nobody has access", organization base permission mislabelled as a team grant, contributors with no access being suggested for removal, and advisory counts being silently zeroed by a second writer.

Unattributed commits are counted and reported, never guessed at. When a commit's author email is not linked to a GitHub account, matching on raw name or email would silently credit the wrong person. The live run found 6 such commits and said so.

Four access paths are stored and reported separately: direct, outside collaborator, team-inherited, and the organization's own base permission. Each has a different remediation — revoking a team grant affects every repo that team can reach; base permission is one org-wide setting — so merging them into a single "permission" column would destroy the distinction that makes the report actionable. The base-permission path was added after a live scan showed it being mislabelled as team-inherited.

Deliberately skipped: commits outside the default branch, PR reviews beyond the first 50 on a single PR, and deploy keys as an access vector. Each is noted where it applies.

Reliability

Idempotent by construction: every table keyed on natural GitHub IDs with `INSERT … ON CONFLICT DO UPDATE`, progress committed per repo so `--resume` continues after a crash. **Verified live** — two full runs left every fact table unchanged.

Rate limits: pre-emptive pausing before the quota runs out, `Retry-After` honoured, and stored ETags so re-runs cost little. **Verified live** — the second run served 8 of 13 requests from 304s, which do not count against quota. A 100-repo org costs roughly 410 REST requests against a 5,000/hour budget, because advisories are fetched org-wide in one call rather than once per repo, and contributions use one GraphQL query per repo rather than one per member.

The findings were acted on

The live scan's advisory half found 32 open Dependabot alerts — 1 critical, 14 high — all in one repository. They are now **all closed**, via seven version pins, verified by a full install and that project's own 148 tests.

The interesting part is *why* they had accumulated: `pillow` was held at 10.4.0 by `streamlit 1.39`, which caps `pillow<11`. Seventeen of the thirty-two alerts were on that one package and none of them were individually fixable — the blocker was a different dependency entirely. A separate automated bump cleared 30 of 32 but stopped two short of what the advisories actually required, which checking resolved versions against each advisory's vulnerable range caught.

Production-readiness

The three things I would add next: an accept/reject feedback loop so the threshold is defensible with evidence rather than argument; tracking access *changes* between runs, since "granted admin last week, never used it" beats any snapshot; and async repo scanning, which is the obvious bottleneck past a few hundred repos.

What to look at

Run it in 30 seconds	`python main.py --demo` — no token, no org, pinned clock, reproducible output
Dashboard	docs/demo_dashboard.html — open straight from a clone
Reasoning and trade-offs	DESIGN_NOTES.md — includes AI-assistance disclosure (§11)
Execution walkthrough	RUN_REPORT.md — captured output, demo *and* live org
Tests	`python -m pytest -q` — 156 tests, ~0.3s, also run in CI on 3.11 and 3.12

What is mocked, and why: only the API responses in `--demo` mode, because private Dependabot advisories need an org with paid security features and genuinely vulnerable dependencies. The fixtures are shaped like real API payloads and run through the *same* parsing, scoring and rendering code as a live scan. Section 8 of the run report is a real scan against a live organization.
